Sentinel AI Security Audit Report

Date: October 26, 2023

Prepared By: Sentinel Security Engineering Division

Subject: Automated Binary Analysis & Vulnerability Assessment

Status: **CRITICAL RISK**

---

1. Executive Summary

Sentinel AI has completed a deep-structure binary audit of the provided executable using our proprietary GNN (Graph Neural Network) compiler wrapper. The analysis utilized the **GATv2 (Graph Attention Network)** model to identify structural anomalies and control-flow patterns indicative of memory safety violations.

Verdict: CRITICAL

The audit identified **4 high-risk functions** exhibiting patterns consistent with **CWE-119** and **CWE-787**. The most severe vulnerability resides in the authentication logic (`vulnerable_login`), which presents a direct path to stack-based buffer exploitation and arbitrary code execution (ACE).

Metric	Value
Total Functions Flagged	4
Highest Confidence Score	86.29%
Primary Threat Category	Memory Safety (Buffer Overflow / OOB Write)
Compliance Violations	SEI CERT C ARR30-C, CWE-787, CWE-120

---

2. Vulnerability Analysis

Finding 1: Stack-Based Buffer Overflow in `vulnerable_login`

- **Model Confidence:** 86.29%
- **Threat ID:** CWE-119 / CWE-120
- **Technical Breakdown:**

The function allocates `0x30` (48 bytes) on the stack (`SUB RSP, 0x30`). However, the local buffer used for input is located at `[RBP - 0x10]`, providing only 16 bytes of space before overwriting the saved Base Pointer (RBP) and Return Address. The call to `0x1400032a8` (likely an unsafe input function like `gets` or `scanf("%s")`) does not perform bounds checking, allowing an attacker to provide a payload longer than 16 bytes to hijack the execution flow.

- **Violated Rule: CERT C ARR30-C**: Do not form or use out-of-bounds pointers or array subscripts.

Remediation & Secure Code Fix

Replace unsafe input functions with bounded alternatives and validate input length.

```
// --- VULNERABLE CONCEPT ---
void vulnerable_login(char* input) {
    char buffer[16];
    strcpy(buffer, input); // DANGEROUS: No bounds check
}

// --- SECURE REMEDIATION ---
#include <string.h>
#include <errno.h>

int secure_login(const char* input) {
    if (input == NULL) return -1;

    char buffer[16];
    // Use strncpy or strlcpy and manually null-terminate
    strncpy(buffer, input, sizeof(buffer) - 1);
    buffer[sizeof(buffer) - 1] = '\0';

    // Better yet: Use safer C11 bounds-checking interfaces
    // if (strncpy_s(buffer, sizeof(buffer), input, _TRUNCATE) != 0) { ... }

    return strcmp(buffer, "SECRET_PASS");
}
```

Finding 2: Complex OOB Write in `.text` (Dispatch Logic)

- **Model Confidence:** 86.18%
- **Threat ID:** CWE-787 (Out-of-bounds Write)
- **Technical Breakdown:**

This function appears to be a dispatch table or a complex switch-case block. While there is a bounds check (`CMP EAX, 0x6`), the subsequent pointer arithmetic in **Basic Block 1** (`LEA RDX, [RAX*0x4]`) and the use of SIMD instructions (`MOVAPS`, `MOVSD`) to move data into `[RSP + 0x20]` through `[RSP + 0x30]` suggests a high risk of "Off-by-One" errors or misalignment. If the index `EAX` is manipulated via an integer overflow before the comparison, the jump table calculation will point to arbitrary memory.

- **Violated Rule: CWE-787**: Out-of-bounds Write.

Remediation & Secure Code Fix

Ensure strict validation of indices and use `snprintf` for formatted data movement.

```
// --- SECURE REMEDIATION ---
void secure_dispatch(unsigned int index, DataStruct* data) {
    // Strict boundary assertion
    if (index >= MAX_HANDLERS) {
        log_security_event("Invalid Dispatch Index");
        return;
    }

    // Ensure data structure is validated before SIMD operations
    if (data != nullptr) {
        // Use safe copy mechanisms
        std::copy(std::begin(data->values), std::end(data->values), safe_dest);
    }
}
```

Finding 3 & 4: Entry Point Anomalies (`WinMainCRTStartup` / `mainCRTStartup`)

- **Model Confidence:** 76.81% / 76.49%
- **Threat ID:** CWE-119
- **Technical Breakdown:**

The GNN flagged the CRT (C Runtime) startup routines. While these are often boilerplate, the model identified suspicious stack frame initialization. In custom-compiled or obfuscated binaries, these entry points are often hooked by malware or packers to initialize heap-spraying buffers or to bypass DEP (Data Execution Prevention). The hardcoded move of `0xff` into `[RBP - 0x4]` followed by a call to `0x140001154` suggests non-standard initialization that could be leveraged for environment-variable-based overflows.

Remediation

- **Compiler Hardening:** Recompile the binary using modern CRT libraries with `/GS` (Buffer Security Check) enabled.
- **Validation:** Ensure that `WinMain` arguments (`lpCmdLine`) are treated as untrusted input.

---

3. General Mitigations & Best Practices

To harden the application against the identified memory safety threats, Sentinel Security recommends the following systemic upgrades:

1. Binary Hardening Flags

Ensure the following flags are enabled during the build process:

- **ASLR (Address Space Layout Randomization):** Prevents attackers from predicting target addresses for jumps.
- **DEP/NX (Data Execution Prevention):** Marks stack and heap memory as non-executable.
- **Stack Canaries (`/GS` or `-fstack-protector-all`):** Places a "cookie" on the stack to detect overflows before function return.

2. Static & Dynamic Analysis

- **Fuzzing:** Implement AFL++ or libFuzzer to test the `vulnerable_login` function with mutated inputs to identify the exact crash offset.
- **SAST Integration:** Integrate static analysis tools (e.g., Clang-Tidy, Coverity) into the CI/CD pipeline to catch CWE-119 violations before compilation.

3. Secure API Migration

Deprecate all "Banned" APIs in the codebase:

- Replace `gets()` with `fgets()`.
- Replace `strcpy()` / `strcat()` with `strncpy()` / `strncat()` or `strlcpy()`.
- Replace `sprintf()` with `snprintf()`.

End of Report

This document is classified as CONFIDENTIAL and is intended solely for the internal use of the development and security teams.